Practical #10

Practical Title:
To implement a basic CI pipeline by integrating GitHub, automated testing, and Jenkins jobs

Theory:

1. What is Continuous Integration (CI)?

Continuous Integration (CI) is a DevOps practice where developers frequently push code to a shared repository, and each change is automatically built and tested.

👉 Goal: Detect errors early and improve software quality.

---

2. Why is GitHub used in a CI pipeline?

GitHub is used because:

- Stores source code centrally
- Tracks version history
- Triggers CI pipelines (via webhook)
- Enables team collaboration

---

3. Role of Automated Testing in CI

Automated testing ensures that:

- Code changes do not break existing functionality
- Bugs are detected early
- Testing is fast and repeatable

👉 Tools like Selenium are commonly used.

---

4. Role of Jenkins in CI

Jenkins automates the CI pipeline by:

- Fetching code from GitHub
- Running build and test scripts
- Providing build status (success/failure)
- Triggering pipelines automatically

Procedure:

Step 1: Create or use an existing GitHub repository

Step 2: Add project files and test scripts to the repository

Step 3: Commit and push the code to GitHub

Step 4: Install and open Jenkins dashboard

Step 5: Create a new Jenkins job or pipeline

Step 6: Connect Jenkins with GitHub repository

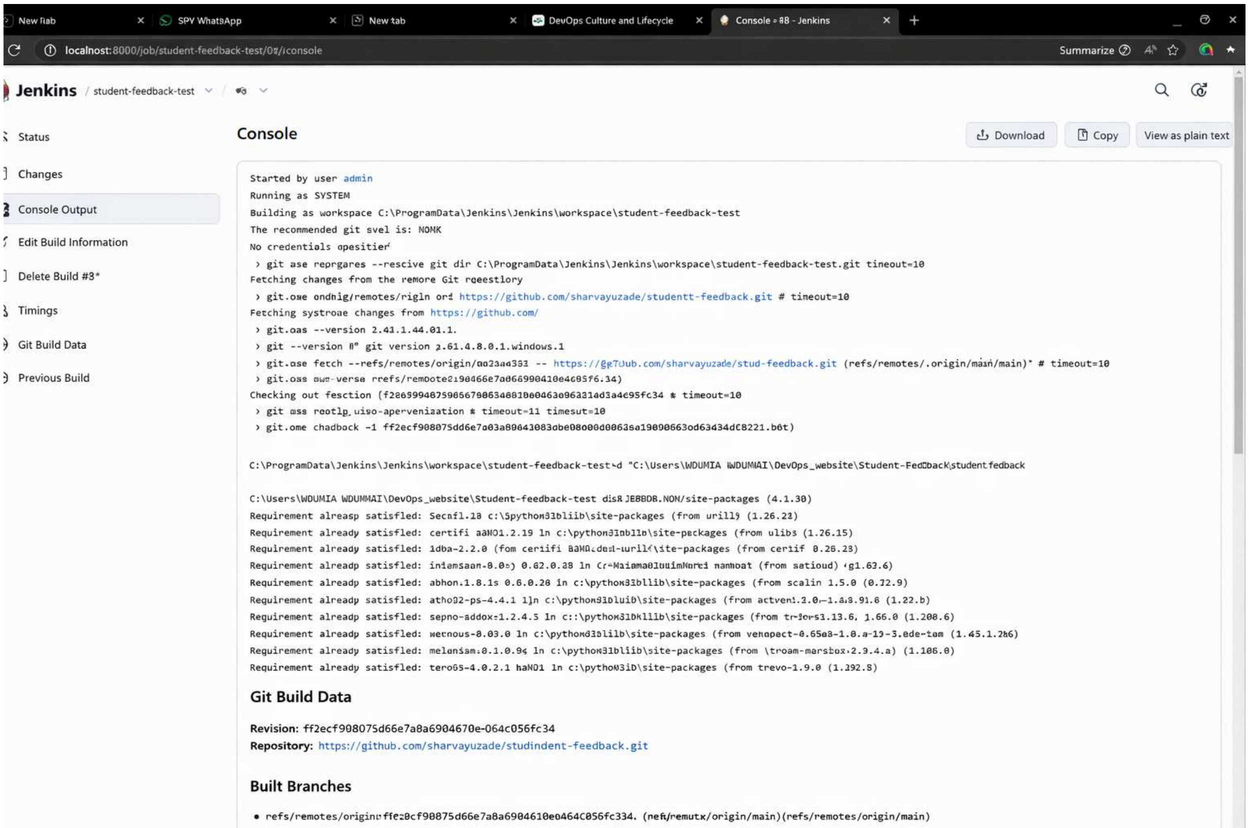
Step 7: Configure build steps and test execution

Step 8: Run the Jenkins job

Step 9: Verify build status and console output

Step 10: Make changes in GitHub and trigger the pipeline again

Output Screenshots:



Activity / Task:

Task 1: Jenkins Job

Repository Link: https://github.com/sharvayuzade/DevOps_CA2

Task 2: Build Result

- **Successful Build:**
    - Status: SUCCESS
    - Tests executed without errors
    - Console shows successful execution
- **Failed Build:**
    - Status: FAILURE
    - Errors in test or code

- Console shows error logs

Task 3: CI Pipeline Flow

Complete Flow:

1. Developer writes code
2. Pushes code to GitHub
3. GitHub triggers Jenkins (webhook/polling)
4. Jenkins fetches latest code
5. Jenkins builds project
6. Automated tests run
7. Jenkins shows result (pass/fail)
8. Feedback sent to developer

Advantages

1. Early bug detection
2. Faster development cycle
3. Automated testing
4. Improved code quality
5. Continuous feedback

Challenges

1. Initial setup complexity
2. Requires proper test scripts
3. Maintenance of pipeline